SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Case Study

COURSE NAME: Computer Networks **COURSE CODE:** CSA0706

COURSE OUTCOME COVERED

CO3: *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

QUESTIONS: 4 Total Marks: 40

Code of Conduct:

I _____V.Lasya(192524186)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.lasya

(For Faculty Use)

Criteria	Understanding the case (2.5)	Application of Concepts (2.5)	Analysis (2)	Solution Design (2)	Clarity (1)	Total (10)
Q1						
Q2						
Q3						
Q4						

Faculty Signature

QUESTIONS: 4 Total Marks: 40

1. Cloud-Based Video Conferencing Quality Issues

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

1. Analyze whether the problem is primarily caused by the transport protocol (TCP/UDP) or the application layer.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

Answer:

1. Transport Protocol vs Application Layer:

• The symptoms described — frozen video frames and delayed communication alongside 8% packet loss — point primarily to a transport-layer issue rather than a purely application-layer one. If the platform is using (or falling back to) TCP for media delivery, TCP's built-in reliability mechanism forces retransmission of every lost packet and delivers data strictly in order; with 8% loss, this causes the receiver to repeatedly stall (Head-of-Line blocking) waiting for missing segments, which manifests exactly as frozen frames and delayed audio/video.

• The 180 ms jitter (variation in packet arrival time) further indicates that whatever buffering strategy is in place at the application layer is not adequately smoothing out this variation, compounding the problem — so both the transport protocol choice and the application's jitter-buffer configuration are contributing factors.

2. Justification Based on Real-Time Multimedia Requirements:

• Real-time multimedia applications prioritize timely delivery over guaranteed delivery — a slightly lost or corrupted audio/video frame is far less disruptive to the user than a delayed one, since the human ear/eye can tolerate small losses but not stalls.

• TCP is designed for the opposite priority (reliability and in-order delivery at the cost of delay), making it fundamentally unsuitable for real-time media under lossy conditions: every lost packet triggers a retransmission and a wait, directly causing the freezing and delay observed.

• UDP, by contrast, does not retransmit or wait for lost packets — it simply delivers what arrives, allowing the application to conceal minor loss (e.g., via error concealment) without stalling the whole stream — which is why real-time voice/video protocols (RTP over UDP) are the industry standard for conferencing platforms.

3. Recommended Improvements:

• Ensure the conferencing platform transmits media over UDP-based RTP/RTCP rather than TCP, so that lost packets are simply skipped instead of stalling the stream.

• Implement Forward Error Correction (FEC) and Packet Loss Concealment (PLC) at the application layer to recover from or mask the 8% packet loss without needing retransmission.

• Use an adaptive jitter buffer that dynamically adjusts its size based on measured jitter (180 ms) to smooth out arrival-time variation while keeping added delay minimal.

• Apply QoS/DiffServ marking to prioritize real-time RTP traffic over the network, and use adaptive bitrate/codec selection (e.g., Opus with in-band FEC) so quality degrades gracefully under congestion rather than freezing.

• Where TCP cannot be avoided (e.g., certain firewall-restricted TURN relays), restrict it to signaling traffic only and keep media strictly on UDP.

2. High DNS Lookup Latency for Global Web Services

A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

1. Analyze the reasons behind the high DNS resolution time.

2. Propose a suitable DNS architecture using techniques such as Anycast DNS, DNS pre-fetching, and TTL optimization to reduce the lookup time below 50 ms.

3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

Answer:

1. Reasons Behind High DNS Resolution Time:

• A single (or very few) authoritative/DNS resolver location means users on other continents must send queries across long-distance links, adding significant round-trip time — consistent with the observed ~800 ms average.

• Low or absent caching (short TTLs or no local caching resolvers) forces repeated full recursive resolution — walking through root, TLD, and authoritative servers — for nearly every request instead of serving from a nearby cache.

• Absence of Anycast routing means every query is sent to the same fixed server location regardless of the user's geographic position, rather than being automatically routed to the nearest available server.

• Server-side load during peak periods can queue incoming queries at an overloaded single DNS server, adding further processing delay on top of network transit time.

2. Proposed DNS Architecture:

• Deploy Anycast DNS: advertise the same DNS server IP address from multiple geographically distributed points of presence (PoPs); BGP automatically routes each user's query to the nearest/least congested instance, sharply cutting round-trip time.

• Enable DNS pre-fetching at the browser/client level so likely-needed domain names are resolved in advance (e.g., for linked resources) before they are actually requested, hiding lookup latency from the user's perceived load time.

• Apply TTL optimization: set a moderate TTL (e.g., 300 seconds) that is long enough to let resolvers and browsers cache results (avoiding repeated full lookups) but short enough to allow reasonably fast failover if a server needs to be taken out of rotation.

• Together, Anycast (reduces per-query travel distance) plus effective caching from tuned TTLs (reduces the number of full lookups needed) plus pre-fetching (hides remaining latency) can realistically bring effective/perceived lookup time below the 50 ms target.

3. Advantages and Limitations of the Proposed Architecture:

• Advantages: Anycast greatly reduces latency by serving users from the nearest node; it also improves availability since if one node fails, BGP automatically reroutes traffic to the next nearest healthy node with no DNS record changes needed; the architecture scales naturally — new PoPs can

be added as global user base grows; pre-fetching improves perceived responsiveness for the end user.

• Limitations: Anycast requires careful BGP configuration and can occasionally suffer from routing anomalies or 'polarization' (all traffic unexpectedly favoring one node); keeping DNS zone data consistent across many distributed authoritative servers adds operational complexity; TTL tuning is a trade-off — too short increases query load and negates caching benefits, too long slows propagation of changes/failover; pre-fetching can generate extra, sometimes unnecessary, DNS traffic for domains that are ultimately not used.

3. Distributed DNS Design for Global Cloud Services

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how TTL values, Anycast routing, and DNS caching influence system performance and fault tolerance.

Answer:

1. Impact of Centralized DNS Architecture:

• A single primary DNS server becomes a single point of failure — if it goes down or is overloaded (as during a promotional traffic spike), all users worldwide are affected simultaneously, with no automatic fallback.

• Because all queries funnel to one fixed location, users far from that server experience high latency purely due to physical distance/network transit time, explaining the observed ~800 ms lookups.

• The architecture cannot distribute load during traffic surges, so queries queue up at the single server, causing the intermittent delays reported during promotional events; overall it is a scalability and fault-tolerance bottleneck.

2. Proposed Distributed DNS Solution:

• Deploy Anycast DNS across multiple globally distributed PoPs so the same DNS IP is announced from many locations, letting BGP route each user to the nearest, least-loaded instance automatically.

• Maintain multiple synchronized authoritative DNS servers (primary/secondary with automated zone transfers) across regions so no single server outage can take down resolution capability.

• Add geo-based/load-balanced DNS resolution to distribute query load intelligently across available nodes, and deploy regional caching resolvers (recursive caches close to users, potentially CDN-integrated) to further cut repeated lookups.

• Implement continuous health-checking with automatic failover, so an unhealthy DNS node is removed from rotation transparently, without any change needed to how clients query the service.

3. Influence of TTL, Anycast Routing, and DNS Caching:

• TTL values: shorter TTLs improve fault tolerance and freshness by forcing faster re-resolution (quicker failover to a healthy server) but increase query load and average latency since caching benefit is reduced; longer TTLs improve performance and reduce server load through better caching but slow down propagation of failover or configuration changes.

• Anycast routing: improves performance by directing each user to the nearest/least congested server, and improves fault tolerance because if one anycast node fails, BGP transparently reroutes traffic to the next nearest healthy node without any DNS record update.

• DNS caching: reduces repeated queries reaching the authoritative servers (lowering both latency and server load), but overly aggressive caching combined with a long TTL can cause clients to keep using a failed server's cached response for longer than desired — so caching, TTL, and Anycast need to be tuned together to balance performance against fault-tolerance responsiveness.

4. TCP Latency Increase in an Electronic Stock Trading Platform

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Analyze three possible TCP-related factors responsible for the latency increase, considering flow control, Nagle's algorithm, and SYN queue limitations.
2. Explain how each factor affects transaction processing in high-frequency systems.

3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

Answer:

1. Three TCP-Related Factors Behind the Latency Increase:

• Flow Control: under the massive burst of orders at market open, the receiver's buffers fill faster than they can be processed, shrinking the advertised receive window (rwnd); this forces the sender to throttle transmission and wait, adding delay to each acknowledgment cycle.

• Nagle's Algorithm: this algorithm batches small outgoing segments together and waits for an acknowledgment before sending the next small packet, to reduce network overhead; since trading orders are typically small messages, Nagle's algorithm delays their transmission — and when combined with the receiver's delayed-ACK mechanism, this interaction is a well-known cause of latency spikes of tens of milliseconds.

• SYN Queue Limitations: the surge of new/reconnecting client connections exactly at market open can exceed the server's SYN backlog queue size, causing incoming SYN packets to be dropped and retransmitted by clients — adding extra round trips to connection establishment before any order can even be acknowledged.

2. Effect of Each Factor on High-Frequency Transaction Processing:

• Flow control throttling causes transmission pauses exactly when order volume is highest, meaning the systems most in need of speed are the ones most delayed — directly increasing acknowledgment latency during the critical market-open window.

• Nagle's algorithm delays the sending of each small order/acknowledgment message while waiting to batch or waiting for an ACK, which is catastrophic for high-frequency trading where every millisecond of delay can affect execution price and profitability.

• SYN queue overflows delay or drop new connection attempts, meaning some clients cannot even establish a session promptly during the highest-demand period, compounding the overall acknowledgment latency seen system-wide.

3. Recommended TCP Configuration Changes:

• Disable Nagle's algorithm on trading connections (set TCP_NODELAY) so small order messages are sent immediately without waiting to batch, and correspondingly disable/reduce delayed ACK

(e.g., enable TCP_QUICKACK) on the server side to avoid the classic Nagle-plus-delayed-ACK interaction.

• Increase socket buffer sizes (SO_RCVBUF/SO_SNDBUF) and enable TCP window scaling so the flow-control window can grow large enough to avoid unnecessary throttling during order bursts.

• Increase the SYN backlog queue size (tune net.core.somaxconn and tcp_max_syn_backlog) and enable SYN cookies so the server can absorb the connection burst at market open without dropping incoming SYNs.

• Use persistent, pre-established connections (connection pooling/keep-alive) for order submission instead of opening a new TCP connection per order, avoiding repeated connection-setup overhead entirely.

• Consider a low-latency congestion control algorithm (e.g., BBR) instead of traditional loss-based algorithms, since loss-based algorithms can overreact to minor congestion and unnecessarily slow down transmission in a low-latency trading environment.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand